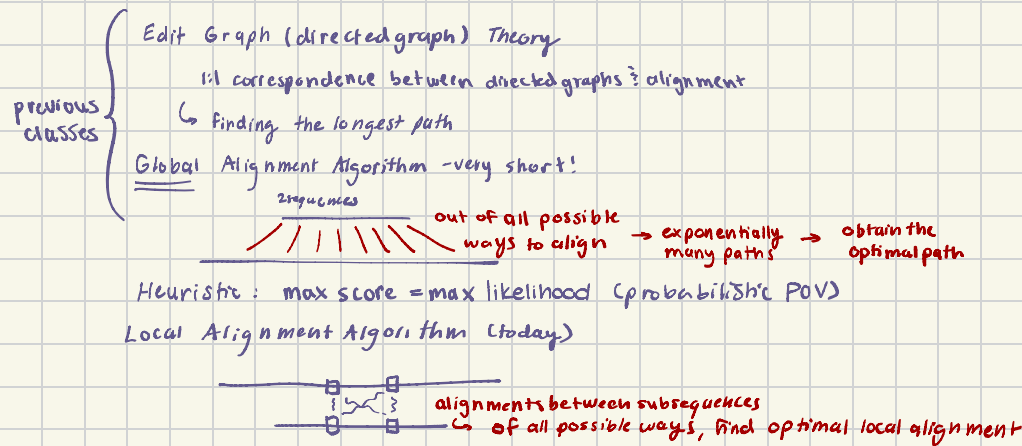


Chp 1: Pairwise Sequence Alignment Algorithms

- 1.1: Edit Graphs, Dynamic Programming, Principle of Optimization
- 1.2: GLOBAL ALIGNMENT ALGORITHM
- 1.3: HEURISTIC Interpretation of Alignment Score
- 1.4: Scoring Scheme for Alignment $\equiv$ Mathematical Models of Evolution
- 1.5: LOCAL ALIGNMENT ALGORITHM $\rightarrow$ most utility in biology!



1.5: Local Alignment Algorithms

Combinatorics of Sequences/Strings

DEF: PREFIXES - all strings formed from string, starting with the first, increment length by one

DEF: SUFFIXES - all strings formed starting from the end of your sequence

DEF: (SUB)Sequences/Strings - any contiguous substring from position n

EXAMPLE sequence: $x = \text{AACTT}$

PREF (x): A

AA
AAC
AACT
AACTT

SUFF (x): T

TT
CTT
ACTT
AACTT

START @: pos 1 pos 2 pos 3 pos 4 pos 5

substring (x):

A	A (2)	C (3)	T (4)	T (5)
AA	AC (2-3)	CT (3-4)	TT (4-5)	
AAC	ACT (2-4)	CTT (3-5)		
AACT	ACTT (2-5)			
AACTT				

PREFIXES SUFFIXES

If you take all the SUFFIXES of PREFIXES, you get all substring $\hat{=}$ vice versa

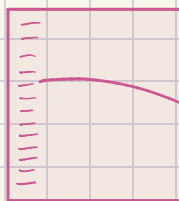
* $\text{SUFFIXES}(\text{PREFIXES}(x)) = \text{SUBSTRINGS}(x)$

* $\text{PREFIXES}(\text{SUFFIXES}(x)) = \text{SUBSTRINGS}(x)$

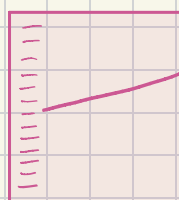
THE SMITH-WATERMAN ALGORITHM

The Local Alignment Problem: GIVEN 2 sequences, $x \hat{=} y$, and the scoring scheme, δ , $(\alpha^*)^*$ $(\beta^*)^*$
 COMPUTE 2 substrings, $\alpha^* \hat{=} \beta^*$ where α^* is a substring of x ; β^* is a substring of y
 whose optimal global alignment of $\alpha^* \hat{=} \beta^*$ is maximum over all pairs
 of substrings of sequences $x \hat{=} y$. That alignment, α^* aligned to β^* ($\alpha^* \hat{=} \beta^*$)
 is the OPTIMAL GLOBAL ALIGNMENT

substring(x) =

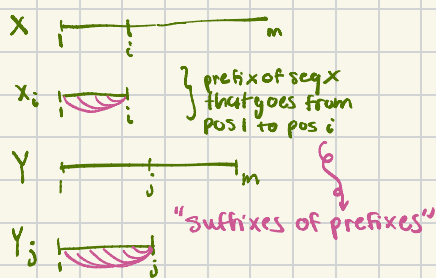


substring(y) =



α^* max score
 β^*

DEF: $v(i,j)$ = value of optimal global alignment between all pairs of suffixes of the prefixes X_i & Y_j



$v(i,j)$ has the same recurrence as the $s(i,j)$ recurrence in global alignment

THE ALGORITHM

OPT-LOCAL-ALIGN

The following algorithm computes the optimal of $x \approx y$: $V(X,Y) = V^{OLA}(X,Y)$

$$v(i,0) = 0, v(0,j) = 0$$

for $i > 0 \approx j > 0$

$$v(i,j) = \max \begin{cases} 0, \\ v(i-1,j-1) + \delta(x_i, y_j) \\ v(i-1,j) + \delta(x_i, -) \\ v(i,j-1) + \delta(-, y_j) \end{cases}$$

* NOTE: - means GAP

$$V^{OLA}(X,Y) = \max [v(i,j), 0 \leq i \leq N, 0 \leq j \leq M]$$

* dynamic programming matrix wr $v(i,j)$ as matrix

OLA is largest entry in the matrix

↳ no longer in bottom right corner

we look throughout the entire matrix

↳ stops @ (substring) MYSTERY, why doesn't it continue?

↳ then you restart

QUADRATIC ($N \times N$)

* used to find preserved regions in proteins, active sites, etc.

A PROOF of the ALGORITHM

Let α be a suffix of X_i and β be a suffix of Y_j :

We define $\alpha^* \approx \beta^*$ as the optimal local alignment (OLA) of $X \approx Y$

Since α or β can be empty, then zero is a value of the alignment

↳ DEF: ϵ : empty string is string with no letters/symbols

Suppose $\alpha^* \approx \beta^*$ are NOT empty strings, then letter $X_{i+1} = x_i$ must be aligned with "—" of with $Y_{j+1} = y_j$

↳ NOTE: $X = X(1)X(2) \dots X(N)$

$$Y = Y(1)Y(2) \dots Y(M)$$

$$X(i) = x_i$$

$$Y(j) = y_j$$

...

PROOF TBF next class

